THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

DSAA2011 Machine Learning

L11 Dimensionality Reduction

Dr. Weikai Yang

Data Science and Analytics Thrust & Computational Media and Arts Thrust
Information Hub
Hong Kong University of Science and Technology (Guangzhou)

April 12, 2026

Syllabus

Lec #	Topic	Lecturer
1	Introduction + Course Info	Zixin
2-4	Supervised learning: regression and classification	Zixin
5	Feature engineering + Model evaluation and choice	Zixin
6	Model evaluation and choice + Feature selection	Zixin
7	Ensemble learning (supervised learning)	Weikai
	Midterm (28 March, 1:30 PM - 3:00 PM): mark it on your calendar!	
8-9	Unsupervised learning: clustering	Weikai
10	**Dimensionality reduction**	Weikai
11	Markov and graphical models	Weikai
12	Active learning + Reinforcement learning	Weikai
13	Project presentation	Both
	Project report submission (TBD)	
	Final exam (TBD)	

Recap: clustering setup

- Clustering is an unsupervised learning task that partitions data into groups without ground-truth labels.
- External metrics use labels when available, such as Purity, ARI, FMI, NMI, and V-measure.
- Internal metrics use only the geometry of the data, such as Silhouette, CH, DB, and Dunn.

Recap: prototype-based methods

- k-Means alternates assignment and update steps to minimize the within-cluster sum of squares.
- k-Means++ improves initialization, and the elbow method gives a practical heuristic for choosing K.
- k-Medoids uses actual data points as centers, so it is more robust to noise and outliers.
- Kernel k-Means extends prototype-based clustering to non-linear structure through kernel similarities.

Recap: density-based methods

- DBSCAN defines clusters based on density reachability.
- DBSCAN distinguishes core points, border points, and noise points.
- OPTICS extends the density idea to varying density levels and reveals structure through a reachability plot.

Recap: hierarchical methods

- Hierarchical clustering builds a dendrogram by repeated merges or splits.
- Single, complete, average, and Ward linkage define different notions of distance between clusters.

Recap: soft clustering with GMMs

- A Gaussian Mixture Model represents data as a weighted sum of Gaussian components.
- The latent variable indicates which component generated each sample.
- The E-step computes responsibilities, which are the soft memberships of samples to components.
- The M-step updates mixture weights, means, and covariances from those responsibilities.

Recap: choosing a clustering method

Method family	Works well when	Main caution
Prototype-based	Clusters are compact and well separated	Needs K and is geometry sensitive
Density-based	Clusters are irregular and noise is present	Parameter choice matters
Hierarchical	Multi-resolution structure is useful	Computation can be expensive
GMM + EM	Soft membership is needed	Initialization can matter a lot

- The right choice depends on cluster shape, noise level, overlap, and whether soft assignments are needed.

Quiz: Clustering

Quiz

B

- When calculating Rand Index, for sample pair x_i, x_j, we have the clustering labels $c_i = c_j$ and ground truth labels $\ell_i \neq \ell_j$, it is viewed as
  - A True positive
  - B False positive
  - C True negative
  - D False negative

Quiz

Which of the following are external clustering metrics

A Rand Index
B Mutual Information
C Silhouette Score
D Davies-Bouldin Index

Quiz

D

- The Rand Index computes the similarity between two clusterings by considering
  - A The distances between cluster centroids
  - B The variance within each cluster
  - C The entropy of cluster assignments
  - D Pairs of points and whether they are clustered together or apart in both clusterings

Quiz

B

- The Silhouette score for a point measures
  - A The distance between cluster centroids
  - B How similar it is to its own cluster compared to other clusters
  - C The total variance within a cluster
  - D The number of points in its cluster

Quiz

A

- What does a high Calinski-Harabasz Index indicate?
  - A Well-separated and compact clusters
  - B Poorly separated clusters
  - C High intra-cluster variance
  - D Uneven cluster sizes

Quiz

C • How does K-means++ improve upon K-means?

A It automatically determines the number of clusters
B It eliminates the need for distance calculations
C It initializes centroids to reduce the risk of poor clustering
D It uses a hierarchical approach to assign points

B • Which of the following statements is true about DBSCAN?

A It requires the user to specify the number of cluster
B It can identify clusters of arbitrary shapes and handle noise points
C It is sensitive to the order of samples and always produces the same result.
D It uses a probabilistic approach to assign samples to clusters.

C

- Which clustering method would be most appropriate for a dataset with noisy points and clusters of varying densities?
  - A Hierarchical clustering with Wards linkage
  - B Gaussian Mixture Models
  - C OPTICS
  - D k-Means

Why dimensionality reduction?

- **Curse of Dimensionality:**

  High-dimensional data leads to sparsity, computational cost, and overfitting
- In high dimensions, distances tend to concentrate and neighborhoods become less informative.

- **Goals:**
  - Visualization (e.g., 100D to 2D)
  - Noise reduction
  - Feature extraction
  - Computational efficiency
  - ...

Dimensionality reduction: big picture

- PCA and classical MDS are unsupervised, linear methods
- Isomap, SNE, t-SNE, and UMAP are unsupervised, nonlinear methods
- LDA is supervised and uses labels to separate classes
- Before choosing a method, ask what should be preserved: variance, distances, neighborhoods, or class separation

Principal Component Analysis (PCA)

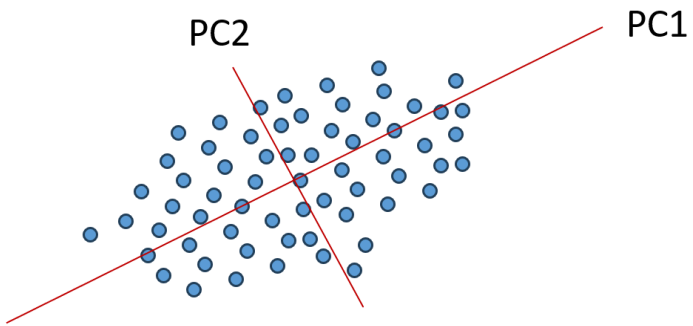
- Principal Component Analysis: an *unsupervised, linear* dimensionality reduction technique

⋆ **How?**

- Finds new axes that capture most important patterns in the data
  - New axes: principal components, linear combinations of existing axes
  - Practical note: center the data first, and standardize when different features use very different scales

⋆ **Output:**

- $\boldsymbol{x}_i \approx \sum_{j=1}^{k} z_{ij}\boldsymbol{v}_j$, $\boldsymbol{v}_j$'s are the best directions (principal components) to capture important patterns



PCA: how to determine new axes?

For centered data, two equivalent viewpoints are useful.

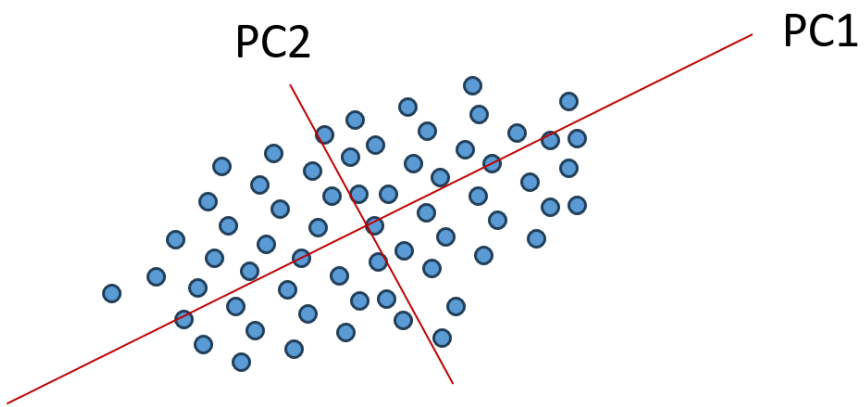
- Maximize variance:

  if many samples are collapsed in the new space, we lose information

- Minimize reconstruction error:

  we can use lower-dimensional representations to approximately reconstruct the original data

equivalent

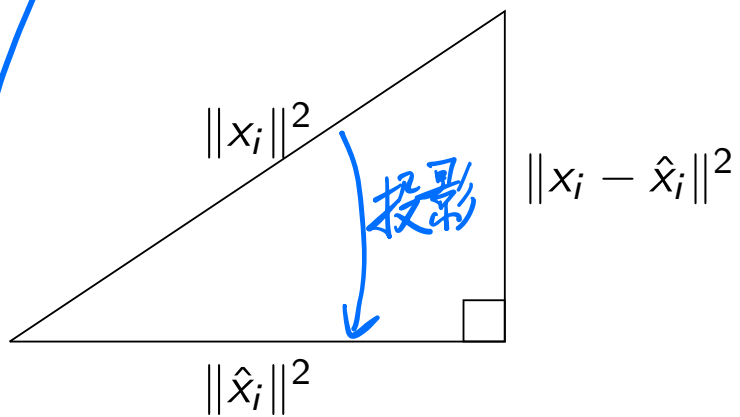


PCA: how to determine new axes?

⊙ **Idea:** Minimize the reconstruction error ⇔ maximize the projected variance

- Data: $X \in \mathbb{R}^{n \times d}$ (n samples, d features)
- Assume we have the projection matrix $U \in \mathbb{R}^{d \times k}$: U contains k **orthonormal** bases that define the subspace
  - Projected points (the coordinates in the subspace): $Z = XU \in \mathbb{R}^{n \times k}$
  - Reconstructed points: $\hat{X} = ZU^\top = XUU^\top$
- By Pythagorean theorem, the reconstruction error

$$\underbrace{\sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2}_{\text{Reconstruction error}} = \underbrace{\sum_{i=1}^{n} \|x_i\|^2}_{\text{Constant}} - \underbrace{\sum_{i=1}^{n} \|\hat{x}_i\|^2}_{\text{Variance}}$$



PCA: a toy example in 3D

- Consider five centered points in $\mathbb{R}^3$:

  $x_1 = (2, -1, 0.1)$, $x_2 = (1, 1, -0.1)$, $x_3 = (0, 2, 0)$,

  $x_4 = (-1, -1, 0.1)$, and $x_5 = (-2, -1, -0.1)$

- These points vary a lot in the x- and y-directions, but only slightly in the z-direction.
- If we project onto the xy-plane, we discard only the small z-coordinates.
- If we project onto the xz-plane, we discard the much larger y-coordinates.

$\Rightarrow$ A good PCA subspace keeps the large spread and discards the small residual direction.

Compare two planes

$$U_1 = \mathrm{span}(e_1, e_2)$$

$$\sum_{i=1}^{5} \|x_i - \hat{x}_i\|^2 = \sum_{i=1}^{5} z_i^2 = 0.04$$

$$U_2 = \mathrm{span}(e_1, e_3)$$

$$\sum_{i=1}^{5} \|x_i - \hat{x}_i\|^2 = \sum_{i=1}^{5} y_i^2 = 8$$

PCA: variance maximization

First, consider projecting the data in one-dimensional space ...

- We are projecting data onto a unit vector $u \in \mathbb{R}^d$: $z = Xu$, and hence $z \in \mathbb{R}^n$
- Variance of projection:

$$\text{Var}(z) = \text{Var}(Xu) = u^T \text{Cov}(X) u = u^T \Sigma u, \text{ where } \Sigma = \frac{1}{n} X^T X$$

- Optimization problem:

$$\max_u u^T \Sigma u, \quad \text{subject to} \quad u^\top u = 1$$

or equivalently

$$\max_u \frac{u^\top \Sigma u}{u^\top u} \tag{2.1}$$

Q: How to solve this? How about projection onto a k ($k \geq 1$)-dimensional space?

Recap: linear algebra

Spectral Decomposition

For an $n \times n$ real symmetric matrix A, we have $A = \sum_{i=1}^{n} \lambda_i e_i e_i^\top$, where λ_i is the i-th eigenvalue, and e_i is the corresponding normalized eigenvector.
Let $\Lambda = \text{diag}\{\lambda_1, \ldots, \lambda_n\}$, $Q = [e_1, \ldots, e_n]$, we have $A = Q\Lambda Q^\top$, and $A^{1/2} = Q\Lambda^{1/2} Q^\top$

Maximization of Quadratic Forms for Points on the Unit Sphere (Rayleigh quotient)

Let B be a positive semi-definite matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$ and associated normalized eigenvectors $e_1, \ldots, e_n$, and u is a unit vector, then

- $\max_{u \neq 0} u^\top B u = \lambda_1$
- $\min_{u \neq 0} u^\top B u = \lambda_n$
- $\max_{u \neq 0, u \perp e_1, \ldots, e_k} u^\top B u = \lambda_{k+1}$

Sketch of proof (optional)

Spectral Decomposition

For an $n \times n$ real symmetric matrix A, we have $A = \sum_{i=1}^{n} \lambda_i e_i e_i^\top$, where λ_i is the i-th eigenvalue, and e_i is the corresponding normalized eigenvector.
Let $\Lambda = \mathrm{diag}\{\lambda_1, \ldots, \lambda_n\}$, $Q = [e_1, \ldots, e_n]$, we have $A = Q\Lambda Q^\top$, and $A^{1/2} = Q\Lambda^{1/2} Q^\top$

Proof.

- For a real symmetric matrix A, the eigenvectors of different eigenvalues are orthogonal (check $e_i^\top A e_j$ and $e_j^\top A e_i$)
- There is an orthogonal matrix Q such that $(AQ = \Lambda Q \Leftrightarrow) Q^{-1} A Q = \Lambda$ and $Q^{-1} = Q^\top$

Sketch of proof (optional)

Maximization of Quadratic Forms for Points on the Unit Sphere

Let B be a positive semi-definite matrix with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n \geq 0$ and associated normalized eigenvectors $e_1, \ldots, e_n$, and u is a unit vector, then

- $\max_{u \neq 0} u^\top B u = \lambda_1$
- $\min_{u \neq 0} u^\top B u = \lambda_n$
- $\max_{u \neq 0, u \perp e_1, \ldots, e_k} u^\top B u = \lambda_{k+1}$

Proof.

- $u^\top B u = u^\top B^{1/2} B^{1/2} u = u^\top Q \Lambda^{1/2} Q^\top Q \Lambda^{1/2} Q^\top u = (Q^\top u)^\top \Lambda (Q^\top u)$
- $v := Q^\top u$ is still a unit vector, and $v^\top \Lambda v = \sum_{i=1}^n \lambda_i v_i^2 \leq \lambda_1$, $v^\top \Lambda v = \lambda_1 \Leftrightarrow v_1 = 1; v_2 = v_3 = \cdots = v_n = 0 \Leftrightarrow u = e_1$
- $u = Qv = v_1 e_1 + v_2 e_2 + \cdots + v_n e_n$, $u \perp e_1 \Rightarrow u^\top e_1 = v_1 = 0$. Similarly, we have $v_2 = \cdots = v_k = 0$, then $v^\top \Lambda v = \sum_{i=k+1}^n \lambda_i v_i^2 \leq \lambda_{k+1}$

PCA: principal components

Principal Components

The eigenvectors of the covariance matrix define the principal components.

- Covariance matrix: $\Sigma = \frac{1}{n} X^T X$ is a symmetric matrix
- Eigenvalue decomposition:

$$\Sigma = U \Lambda U^T,$$

  where:
  - $U \in \mathbb{R}^{p \times p}$: matrix of eigenvectors (principal components)
  - Λ: diagonal matrix of eigenvalues ($\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p$)
- Top k components: select columns of $U_k \in \mathbb{R}^{p \times k}$ corresponding to the k largest eigenvalues
- Projection: new data in lower dimensions:

$$Z = X U_k, \quad Z \in \mathbb{R}^{n \times k}$$

- **Variance explained:** $\sum_{i=1}^{k} \lambda_i / \sum_{i=1}^{p} \lambda_i$ variance explained=选的k个特征值之和/所有特征值之和

选对应k largest λ的eigenvectors

↳特征向量(orthonormal)

PCA: algorithm

$x' = \frac{x-\mu}{\sigma}$

1. **Input**: $X \in \mathbb{R}^{n \times p}$ (n samples, p features)
2. **Center the data**: let $\boldsymbol{X}'$ denote the centered data, and standardize it to unit variance when different features use different scales
3. **Compute covariance matrix**: calculate $\Sigma = \frac{1}{n} \boldsymbol{X}'^{\top} \boldsymbol{X}'$
4. **Eigenvalue decomposition**: find eigenvectors (principal components) and eigenvalues (variance) of Σ
5. **Select principal components**: choose the smallest k that meets your goal, such as a scree-plot elbow or 90%–95% cumulative explained variance $\Rightarrow W$
6. **Project data**: transform data to lower-dimensional space: $\boldsymbol{Z} = \boldsymbol{X}'\boldsymbol{W}$, where W is the matrix of selected eigenvectors

PCA: algorithm design

⊙ **Target:** dimensionality reduction

⇐ Minimize reconstruction error

⇐ Maximize projected variance (recall Pythagorean theorem)

$$\underbrace{\sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2}_{\text{Reconstruction error}} = \underbrace{\sum_{i=1}^{n} \|x_i\|^2}_{\text{Constant}} - \underbrace{\sum_{i=1}^{n} \|\hat{x}_i\|^2}_{\text{Variance}}$$

⇐ **What PCA really does:** find principal components (via eigenvalue decomposition)

PCA: how to select k

- Visualization: you might only use $k = 2$ or 3
- Compression or denoising:
  - Use a scree plot to look for an elbow in the eigenvalues
  - Use cumulative explained variance and keep the smallest k that reaches the desired threshold

PCA: more quick questions

Q1. Why do we (usually) need to do standardization?

- Centering: If not centering data around the origin, make sure you use the covariance matrix $(X-\mu)^\top(X-\mu)$ instead of $X^\top X$
- Scaling: If not scaling each coordinate appropriately, the coordinate with the higher variance will dominate the principal component

$\Rightarrow$ PCA results will be sensitive to the units of each coordinate (e.g., height in m and cm)

Q2. Why we use $\Sigma = \frac{1}{n}X^\top X$ instead of $\frac{1}{n-1}X^\top X$?

- It does not matter, the eigenvalues and eigenvectors are the same.

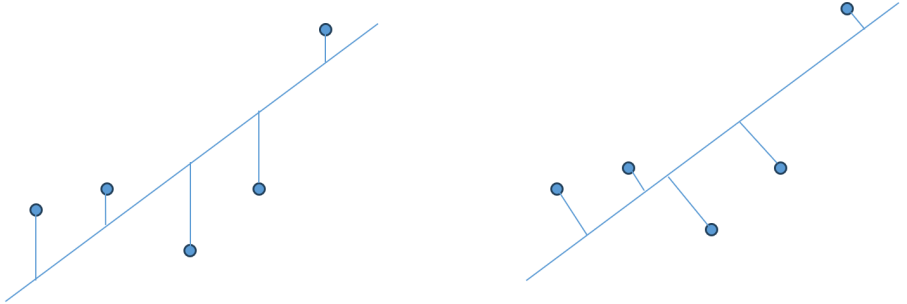
PCA: more quick questions

Q3. What is the difference between linear regression and PCA?

- Both of them have concepts like projection matrix, best subspace, etc.

(a)
- PCA: all dimensions are treated equally
- Linear regression: y is special

(b)
- PCA: minimize the sum of the squared **perpendicular** distances after projecting X into the subspace spanned by the principal components
- Linear regression: minimize the sum of squared errors after projecting y into the subspace spanned by X



PCA: more quick questions

Q4. What would happen if the matrix X is not full rank?

- Let $r = \text{rank}(X)$. Then $\text{rank}(X^\top X) = r$, so the covariance matrix $\Sigma = \frac{1}{n} X^\top X$ has only r non-zero eigenvalues.

(a) If $n > p > r$:

some features are linearly dependent, so Σ is singular

PCA is still well-defined, but only the first r principal components have non-zero variance

the remaining $p - r$ directions have eigenvalue 0, so they add no new information

(b) If $n < p$:

full column rank is impossible, and $\text{rank}(X) \leq n$

after centering, usually $\text{rank}(X) \leq n - 1$, so we can have at most $n - 1$ non-zero principal components

in practice, it is often better to compute PCA via SVD or via the smaller matrix $XX^\top$

Kernel PCA

- Kernel PCA applies PCA in a nonlinear feature space $\phi(x)$
- Using the kernel trick, we work with inner products $K_{ij} = \phi(x_i)^\top \phi(x_j)$ instead of forming $\phi(x)$ explicitly
- The choice of kernel determines which nonlinear patterns can be captured

Q: How to compute its eigenvalues and eigenvectors using kernels $K_{ij} = \phi(x_i)^\top \phi(x_j)$?

- An eigenvector should satisfy $\frac{1}{n}\sum_{i=1}^{n} \phi(x_i)\phi(x_i)^\top v = \lambda v$
- Since $\phi(x_i)^\top v$ is a scalar, we know that v is a linear combination of $\phi(x_i)$, i.e., there exists $\beta \in \mathbb{R}_{n\times 1}$ such that $v = [\phi(x_1), \phi(x_2), \ldots, \phi(x_n)]\,\beta = \phi(X)\beta$
- Let K_c be the centered kernel matrix, then we have $K_c^2\alpha = n\lambda K_c\alpha$

$\Rightarrow$ Solution: solve the eigenvalue problem for $K_c\alpha = n\lambda\alpha$

- Normalize each PC by setting $\alpha_j \leftarrow \alpha_j/\sqrt{\lambda_j}$
- The projected training dataset is $K_c\alpha[:, 1:k]$

PCA: pros & cons

Pros:

- Linear method, simpler to implement
- Computationally efficient for large datasets
- Reduces multi-collinearity effectively

简单高效，能减少多重共线性

Cons:

- Assumes linear relationships in data
- Less effective for preserving local structure
- May lose critical information if variance is not representative
- Sensitive to outliers (variants: robust PCA)

Multidimensional Scaling (MDS)

⊙ **Idea:** MDS aims to preserve pairwise proximities in a low-dimensional embedding

- MDS is especially useful when we know pairwise distances or dissimilarities but do not want to work directly with the original features
- The proximity matrix, such as similarity matrix, distance matrix, and dissimilarity matrix, is unchanged by shifting, rotation, or reflection

⊙ **Variants:**

- **Classical MDS**: closed-form method for Euclidean distances

  For non-Euclidean distance, it can still be applied as an approximation method when the distance structure is sufficiently close to Euclidean
- **Metric MDS**: iterative method that matches metric distances by minimizing stress
- **Non-metric MDS**: iterative method that preserves only the rank order of dissimilarities

Classical MDS

- **Input:** Proximity matrix $\boldsymbol{D} \in \mathbb{R}^{n \times n}$, where d_{ij} is the distance between samples i and j
- **Goal:** Find low-dimensional embedding $\mathbf{Y} \in \mathbb{R}^{n \times k}$ s.t. $\|\mathbf{y}_i - \mathbf{y}_j\| \approx d_{ij}$
  - The original feature vectors are optional once the pairwise distance matrix is available

- Two steps in classical MDS:

1. $\boldsymbol{D} \to \boldsymbol{B}$: double centering converts the squared distance matrix into the centered Gram matrix $\mathbf{B}$

   This step fixes the centroid at the origin
2. $\boldsymbol{B} \to \boldsymbol{E}\boldsymbol{\Lambda}\boldsymbol{E}^\top = \mathbf{Y}\mathbf{Y}^\top$: eigenvalue decomposition of $\mathbf{B}$ returns coordinates up to rotation or reflection

Classical MDS: unique solution

Let $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n \in \mathbb{R}^d$ denote the original high-dimensional data samples

- $d_{ij}^2 = (\boldsymbol{x}_i - \boldsymbol{x}_j)^\top (\boldsymbol{x}_i - \boldsymbol{x}_j) = \boldsymbol{x}_i^\top \boldsymbol{x}_i + \boldsymbol{x}_j^\top \boldsymbol{x}_j - 2\boldsymbol{x}_i^\top \boldsymbol{x}_j$
  - If we learn the inner product, we learn the Euclidean distance
- Let $\boldsymbol{B} = \boldsymbol{X}\boldsymbol{X}^\top$, i.e., $b_{ij} = \boldsymbol{x}_i^\top \boldsymbol{x}_j$, we have $d_{ij}^2 = b_{ii} + b_{jj} - 2b_{ij}$
- **Issue:** there can be multiple DR solutions since we can freely shift or rotate X

D ⇔ B

Q: How can we get a "determined" result?

A: We impose $\sum \boldsymbol{x}_i = \boldsymbol{0}$, i.e., we use double centering

- Double centering recovers the centered Gram matrix, not an absolutely unique coordinate system

Classical MDS: proximity matrix D and inner product matrix B

- With the constraints $\sum \boldsymbol{x}_i = \mathbf{0}$ (or $\sum_{i=1}^n x_{ip} = 0, 1 \le p \le d$), we have

$$\sum_{i=1}^n b_{ij} = \sum_{i=1}^n \boldsymbol{x}_i^\top \boldsymbol{x}_j = (\sum_{i=1}^n \boldsymbol{x}_i)^\top \boldsymbol{x}_j = 0, \quad \sum_{j=1}^n b_{ij} = \sum_{j=1}^n \boldsymbol{x}_i^\top \boldsymbol{x}_j = \boldsymbol{x}_i^\top \sum_{j=1}^n \boldsymbol{x}_j = 0$$

$$\Rightarrow \sum_{i=1}^n d_{ij}^2 = \sum_{i=1}^n (b_{ii} + b_{jj} - 2b_{ij}) = \sum_{i=1}^n b_{ii} + n b_{jj} := \mathrm{Tr}(B) + n b_{jj} \tag{3.1}$$

$$\sum_{i=1}^n \sum_{j=1}^n d_{ij}^2 = n\mathrm{Tr} + \sum_{j=1}^n n b_{jj} = 2n\mathrm{Tr}(B) \tag{3.2}$$

- Recall that $d_{ij}^2 = b_{ii} + b_{jj} - 2b_{ij}$, we have

$$b_{ij} = -\frac{1}{2}\left(d_{ij}^2 - d_{i\cdot}^2 - d_{\cdot j}^2 + d_{\cdot\cdot}^2\right)$$

where $d_{i\cdot}^2 = \sum_j d_{ij}^2/n$, $d_{\cdot j}^2 = \sum_i d_{ij}^2/n$, $d_{\cdot\cdot}^2 = \sum_i \sum_j d_{ij}^2/n^2$

Classical MDS: proximity matrix D and inner product matrix B

- ... we have

$$b_{ij} = -\frac{1}{2}\left(d_{ij}^2 - d_{i\cdot}^2 - d_{\cdot j}^2 + d_{\cdot\cdot}^2\right)$$

where $d_{i\cdot}^2 = \sum_j d_{ij}^2/n$, $d_{\cdot j}^2 = \sum_i d_{ij}^2/n$, $d_{\cdot\cdot}^2 = \sum_i \sum_j d_{ij}^2/n^2$

- Equivalent to: $\boldsymbol{B} = -\frac{1}{2}\boldsymbol{J}\boldsymbol{D}^{(2)}\boldsymbol{J}$, where $\boldsymbol{J} = \boldsymbol{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$

$$\begin{aligned}\boldsymbol{J}\boldsymbol{D}^{(2)}\boldsymbol{J} &= (\boldsymbol{D}^{(2)} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top\boldsymbol{D}^{(2)})(\boldsymbol{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top) \\ &= (\boldsymbol{D}^{(2)} - \frac{1}{n}\mathbf{1}\mathbf{1}^\top\boldsymbol{D}^{(2)} - \frac{1}{n}\boldsymbol{D}^{(2)}\mathbf{1}\mathbf{1}^\top + \frac{1}{n^2}\mathbf{1}\mathbf{1}^\top\boldsymbol{D}^{(2)}\mathbf{1}\mathbf{1}^\top)\end{aligned}$$

$D^{(2)}$：每个元素都是 b_{ij}^2

J：centering matrix

1：全1向量 $\in R^n$

$11^\top$：全1矩阵 $\in R^{n\times n}$

Note: We have $\boldsymbol{B} = -\frac{1}{2}\boldsymbol{J}\boldsymbol{D}^{(2)}\boldsymbol{J}$, which gives the centered Gram matrix determined by $\boldsymbol{D}$

- $D_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|^2$ and $\mathbf{B} = \mathbf{X}\mathbf{X}^\top$ after double centering
- Distances and inner products become equivalent descriptions once points are centered
- If the distances are truly Euclidean, the coordinates are recovered up to translation, rotation, and reflection
- Negative eigenvalues of B indicate that the distances are not exactly Euclidean

Classical MDS: from B to coordinates Y

- Now the remaining question is: how do we find low-dimensional coordinates $\mathbf{Y} \in \mathbb{R}^{n \times k}$?
- If $\boldsymbol{y}_i \in \mathbb{R}^k$ is the coordinate of sample i, then its inner product matrix is

$$\mathbf{Y}\mathbf{Y}^\top = \begin{bmatrix} \boldsymbol{y}_1^\top \boldsymbol{y}_1 & \cdots & \boldsymbol{y}_1^\top \boldsymbol{y}_n \\ \vdots & \ddots & \vdots \\ \boldsymbol{y}_n^\top \boldsymbol{y}_1 & \cdots & \boldsymbol{y}_n^\top \boldsymbol{y}_n \end{bmatrix}$$

$\Rightarrow$ So after recovering $\mathbf{B}$, we want to find $\mathbf{Y}$ such that $\mathbf{B} \approx \mathbf{Y}\mathbf{Y}^\top$

- Suppose $\mathbf{B} = \mathbf{E}\Lambda\mathbf{E}^\top$, where $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$
- If $\mathbf{B}$ is positive semidefinite and we keep the top k positive eigenvalues, then

$$\mathbf{Y} = \mathbf{E}_k \Lambda_k^{1/2}$$

$\Lambda_k^{\frac{1}{2}}$：开根 $\sqrt{\lambda_i}$，$i = 1 \ldots k$

gives $\mathbf{Y}\mathbf{Y}^\top = \mathbf{E}_k \Lambda_k \mathbf{E}_k^\top$, the best rank-$k$ approximation of $\mathbf{B}$ (under the Frobenius norm)

- This is why classical MDS has a closed-form solution

MDS: strain function

- The quality of the MDS solution is evaluated using a **strain** function:

$$\text{Strain}(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n) = \sqrt{\sum_{i<j} (b_{ij} - \boldsymbol{y}_i^\top \boldsymbol{y}_j)^2 / \sum_{i<j} b_{ij}^2}$$

- Lower strain indicates a better fit
- Classical MDS provides the closed-form solution for strain minimization

Classical MDS: algorithm

distance

1. Compute the squared proximity matrix $D^{(2)} = (d_{ij}^2)$ element-wise square
2. Compute the inner product matrix B using double centering: $B = -\frac{1}{2} J D^{(2)} J$, where $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$,
3. Compute the top k positive eigenvalues $\lambda_1, \ldots, \lambda_k$ and corresponding eigenvectors $e_1, \ldots, e_k$ of B, where $B = E_n \Lambda_n E_n^\top$
4. Obtain the projected coordinates $Y = E_k \Lambda_k^{1/2}$, where $E_k = [e_1, \ldots, e_k] \in \mathbb{R}^{n \times k}$ and $\Lambda_k^{1/2} = \mathrm{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_k}) \in \mathbb{R}^{k \times k}$

Q: See the difference from PCA after having eigenvectors? (PCA: $Y = XU_k$)

A: Doing the same as in PCA will lead to the "projected matrix of pair distances"!

We care about coordinates!

Classical MDS: limitations

Classical MDS: find $\boldsymbol{y}_i$ that gives $d_{ij} \approx \hat{d}_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\|_2$ by matching the inner product

- **Limitations:**
  - Assume the distance d_{ij} should be Euclidean distance (or at least "look like" Euclidean distance)

    In other words, when the measure of dissimilarity d_{ij} is more general, classical MDS may be inappropriate or fail
  - Does not support different weights

Why dont we directly find the coordinates $\mathbf{Y}$ that has the smallest distance error?

↳ metric MDS

Metric MDS

- **Solutions:**
  - Directly match the distance by optimizing

$$\text{Stress}(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n) = \sqrt{\sum_{i<j} w_{ij}(d_{ij} - \|\boldsymbol{y}_i - \boldsymbol{y}_j\|)^2 / \sum_{i<j} \|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2}$$

    or sometimes

$$\text{Stress}(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n) = \sqrt{\sum_{i<j} (d_{ij} - \|\boldsymbol{y}_i - \boldsymbol{y}_j\|)^2 / \sum_{i<j} d_{ij}^2}$$

  - Cons: No closed-form solution; it is solved iteratively and can be sensitive to initialization
  - Also called Stress majorization [Link] and widely used in graph drawing
  - Of course, you can also solve it using gradient descent

Non-Metric MDS

- In some cases, the dissimilarities d_{ij} are qualitative

- **Example:** only the rank matters, so the ordering of dissimilarities should stay consistent after embedding
  - In this setting, the exact magnitudes of the dissimilarities are not trusted

$$\text{Stress}(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n, f) = \sqrt{\sum_{i<j} \left(f(d_{ij}) - \|\boldsymbol{y}_i - \boldsymbol{y}_j\|\right)^2 / \sum_{i<j} \|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2}$$

  - f is a monotonically increasing function, i.e., $d_{ij} < d_{kl} \Leftrightarrow f(d_{ij}) < f(d_{kl})$
  - Algorithm:

1. Monotonic regression: update f to minimize the stress function
2. Update $\boldsymbol{y}_1, \ldots, \boldsymbol{y}_n$ to minimize the stress function
3. Repeat these two steps alternatively

MDS: pros & cons

Pros:

- Preserve pairwise distances (Euclidean or other metrics) as closely as possible
- Flexible: work with any dissimilarity matrix, not just Euclidean distances

Cons:

- Assume linear relationships, failing to capture nonlinear manifold structures
- Computationally intensive for large datasets (e.g., eigenvalue decomposition)
- Sensitive to noise and outliers in the dissimilarity matrix
- Results depend on the choice of distance metric and initialization

Comparison between PCA and MDS

PCA:

- Maximize the projected variance
- Compute covariance matrix

MDS:

- Preserve proximity matrix
- Compute inner product matrix

Remarks

- **Centering** (mean centering): mean$= 0$

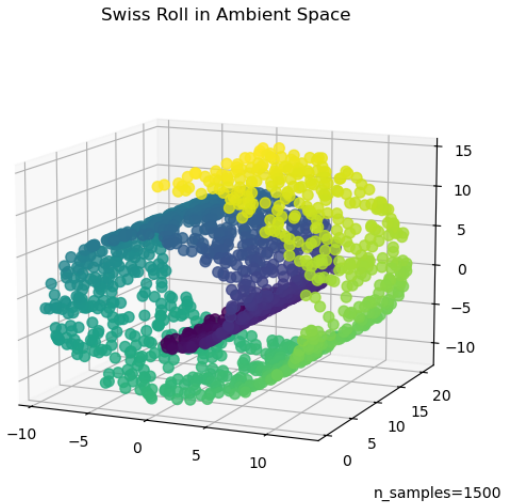
  No scaling, ...
- **Normalization** (min-max scaling): rescale the range, usually $[0, 1]$

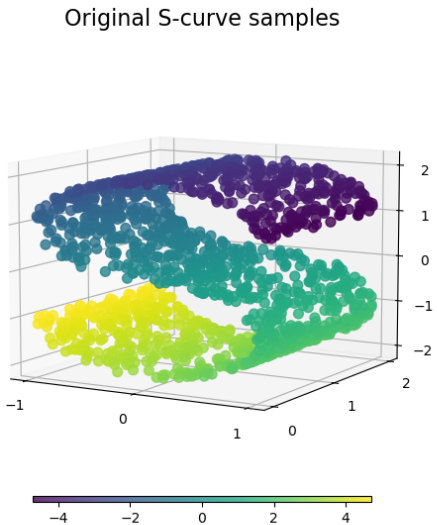
  Sensitive to outliers, ...
- **Standardization** (Z-score normalization): mean$= 0$, standard deviation$= 1$

  Assuming normal distribution, ...
- **Double centering**: used for MDS, less common

Introduction to Isomap

- Euclidean distance used in MDS might not be suitable for samples on a manifold





- How to better capture the distance on a manifold?
- Uses geodesic distances (shortest paths on the manifold) instead of Euclidean distances to capture nonlinear structures.
  - Recipe: build a neighborhood graph, estimate geodesic distances by shortest paths, and then apply classical MDS

Isomap: algorithm

1. **Construct neighborhood graph**: Build a graph where each data point is connected to its k nearest neighbors or points within radius ϵ
2. **Compute geodesic distances**: Estimate geodesic distances between all pairs of points using shortest paths (e.g., Dijkstra's algorithm)
3. **Apply MDS**: Use classical MDS to embed the data into a lower-dimensional space, preserving geodesic distances
4. **Output**: Low-dimensional coordinates that reflect the manifold's geometry

Isomap: pros & cons

Pros:

- Capture nonlinear manifold structures effectively

  (in other words) preserve global geometric properties via geodesic distances

Cons:

- Sensitive to the choice of k or ϵ for neighborhood graph
- Computationally expensive for large datasets (shortest path calculations)
- Assume data lies on a single, well-sampled manifold
- May fail if the manifold has holes or is noisy

LDA: motivation

Linear Discriminant Analysis (LDA)

- Assume we have a dataset $\boldsymbol{X}$ and the associated class labels $\boldsymbol{Y}$
- LDA is a supervised dimensionality-reduction method, so the labels matter
- How can we leverage the class labels for better dimensionality reduction?

- **Goal:** Project data to lower dimensions while maximizing between-class separation relative to within-class variation
  - With C classes, LDA can provide at most $C - 1$ discriminant directions

Separation for two classes

- Assume we are projecting samples into one-dimensional space
- Projections of samples are $a_i = v^\top x_i$, where $v^\top \in \mathbb{R}^d$ is a unit vector

Q: How do we quantify the separation between two clusters?

- One naïve idea: maximize the distance between projected class centers

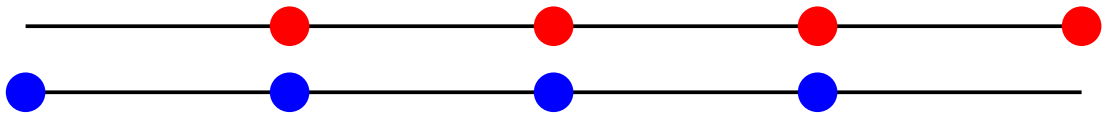
$$\mu_1 = \frac{1}{n_1} \sum_{i \in C_1} a_i = \frac{1}{n_1} \sum_{i \in C_1} v^\top x_i = v^\top \sum_{i \in C_1} \frac{1}{n_1} x_i := v^\top m_1,$$
$$\mu_2 = v^\top m_2$$

Is $|\mu_1 - \mu_2|$ a sufficiently good measure?

Separation for two classes

Q: How do we quantify the separation between two clusters?

- Is $|\mu_1 - \mu_2|$ a good measure?



- We should also consider the variance of the projected classes!

$$s_1^2 = \sum_{i \in C_1} (a_i - \mu_1)^2, \quad s_2^2 = \sum_{i \in C_2} (a_i - \mu_2)^2$$

$\Rightarrow$ "Maximize center gap and minimize variance"

$$\max_{v:\|v\|=1} \frac{(\mu_1 - \mu_2)^2}{s_1^2 + s_2^2}$$

LDA: between/within-class scatter matrix

1. (Numerator) **between-class scatter matrix:** $S_b = (m_1 - m_2)(m_1 - m_2)^\top \in \mathbb{R}^{d\times d}$

   Case: $k = 1$ $\quad (\mu_1 - \mu_2)^2 = (v^\top(m_1 - m_2))^2 = v^\top(m_1 - m_2)(m_1 - m_2)^\top v := v^\top S_b v$

2. (Denominator) **total within-class scatter matrix:** $S_w = S_1 + S_2$

   - Class 1:

$$s_1^2 = \sum_{i\in C_1}(a_i - \mu_1)^2 = \sum_{i\in C_1}(v^\top x_i - v^\top m_1)^2 = v^\top \left(\sum_{i\in C_1}(x_i - m_1)(x_i - m_1)^\top\right) v := v^\top S_1 v$$

- $S_1 = \sum_{i\in C_1}(x_i - m_1)(x_i - m_1)^\top \in \mathbb{R}^{d\times d}$ is called the within-class scatter matrix
- $s_1^2 + s_2^2 = v^\top(S_1 + S_2)v := v^\top S_w v$
- Assume that S_w is invertible

LDA: eigenvalue decomposition

- **Target:**

$$\max_{v:\|v\|=1} \frac{v^\top S_b v}{v^\top S_w v}$$

- **Claim:** v^* should be the eigenvector corresponding to the largest eigenvalue of $S_w^{-1} S_b$
- Why? (proof sketch:)

$$J(v) := \frac{v^\top S_b v}{v^\top S_w v} \Rightarrow \frac{\partial J(v)}{\partial v} = \frac{2[(v^\top S_w v) S_b v - (v^\top S_b v) S_w v]}{(v^\top S_w v)^2}$$

$$\Leftrightarrow (v^\top S_w v) S_b v = (v^\top S_b v) S_w v \Leftrightarrow S_b v = \frac{v^\top S_b v}{v^\top S_w v} S_w v = J(v) S_w v$$

- **What LDA does:** solve the generalized eigenvalue problem $S_b v = \lambda S_w v$
- From another perspective:

$$\lambda v_1 = S_w^{-1} S_b v_1 = S_w^{-1} (m_1 - m_2)(m_1 - m_2)^\top v_1 \propto S_w^{-1} (m_1 - m_2)$$

LDA: eigenvalue decomposition

- **Target:**

$$\max_{v:\|v\|=1} \frac{v^\top S_b v}{v^\top S_w v}$$

- **What LDA does:** solve the generalized eigenvalue problem $S_b v = \lambda S_w v$

Q: Can we get the second eigenvector?

A: S_w invertible $\Rightarrow$ $\text{Rank}(S_w^{-1} S_b) = \text{Rank}(S_b)$

- $\text{Rank}(S_b) \leq \text{Rank}(m_1 - m_2) = 1$
- $\text{Rank}(S_b) > 0$

$\Rightarrow$ $\text{Rank}(S_b) = 1$ $\Rightarrow$ NO in two-class case!

LDA: multiclass extension

- When we have $C \geq 3$ classes, how can we measure their separation?
- Each class center should be far away from each other

$$\sum_{j=1}^{C} n_j(\mu_j - \mu)^2, \quad \text{where } \mu = \frac{1}{n}\sum_{j=1}^{C} n_j \mu_j$$

$$\sum_{j=1}^{C} n_j(\mu_j - \mu)^2 = \sum_{j=1}^{C} n_j v^\top (m_j - m)(m_j - m)^\top v = v^\top S_b v$$

- Each class should be as compact as possible

$$\sum_{j=1}^{C} s_j^2 = v^\top \sum_{j=1}^{C} S_j v = v^\top S_w v$$

$\Rightarrow$ Similarly, we solve a generalized eigenvalue problem $S_b v = \lambda S_w v$

Now $\text{Rank}(S_b) \leq C - 1$ since $m_i - m$'s are linearly dependent

LDA: revisit two-class case

- Use the multiclass LDA with $C = 2$, we have

$$\sum_{j=1}^{2} n_j(\mu_j - \mu)^2 = \frac{n_1 n_2}{n}(\mu_1 - \mu_2)^2$$

$$\sum_{j=1}^{2} n_j(m_j - m)(m_j - m)^\top = \frac{n_1 n_2}{n}(m_2 - m_1)(m_2 - m_1)^\top$$

$\Rightarrow$ **Equivalent!**

LDA: algorithm

1. **Input:** dataset $\boldsymbol{D} \in \mathbb{R}^{n \times d}$ and the associated class labels $\boldsymbol{Y}$ of C classes
2. **Compute** $S_w = \sum_{j=1}^{C} \sum_{i \in C_j} (x_i - m_j)(x_i - m_j)^\top$ and $S_b = \sum_{j=1}^{C} n_j (m_j - m)(m_j - m)^\top$, where C_j is the index set of class j, $n_j = |C_j|$ is the number of samples in class j, $m_j = \frac{1}{n_j} \sum_{i \in C_j} x_i$ is the mean of class j, and $m = \frac{1}{n} \sum_{i=1}^{n} x_i$ is the mean of all samples
3. Solve the **generalized eigenvalue problem** $S_b v = \lambda S_w v$ to find all eigenvalues and their associated eigenvectors $V = [v_1, \ldots, v_k]$
4. **Output:** Projected data $Y = XV \in \mathbb{R}^{n \times k}$

LDA: a statistical perspective (optional)

- Assume we have multiple Gaussian distributions, the prior $P(Y = k) = \pi_k$, and $P(X = x|Y = k) = f_k(x) = \mathcal{N}(x \mid m_k, \Sigma)$
- We can obtain

$$P(Y = k|X = x) \propto \pi_k \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\left\{-\frac{1}{2}(x - m_k)^\top \Sigma^{-1}(x - m_k)\right\}$$

$$\log P(Y = k|X = x) = \text{Constant} + \log \pi_k - \frac{1}{2}(x - m_k)^\top \Sigma^{-1}(x - m_k)$$

- The decision boundary is a linear equation in x:

$$\log P(Y = k|X = x) = \log P(Y = \ell|X = x)$$

$$\log \pi_k - \frac{1}{2} m_k^\top \Sigma^{-1} m_k + x^\top \Sigma^{-1} m_k = \log \pi_\ell - \frac{1}{2} m_\ell^\top \Sigma^{-1} m_\ell + x^\top \Sigma^{-1} m_\ell$$

LDA: a statistical perspective (optional)

- The decision boundary is a linear equation in x:

$$\log P(Y = k|X = x) = \log P(Y = \ell|X = x)$$

$$\log \pi_k - \frac{1}{2} m_k^\top \Sigma^{-1} m_k + x^\top \Sigma^{-1} m_k = \log \pi_\ell - \frac{1}{2} m_\ell^\top \Sigma^{-1} m_\ell + x^\top \Sigma^{-1} m_\ell$$

- Assume we have two classes, and $\pi_1 = \pi_2 = 1/2$, then

$$x^\top \Sigma^{-1}(m_1 - m_2) = \frac{1}{2} m_1^\top \Sigma^{-1} m_1 - \frac{1}{2} m_2^\top \Sigma^{-1} m_2 = (m_1 - m_2)^\top \Sigma^{-1}(m_1 + m_2)/2$$

- Project samples to $\Sigma^{-1}(m_1 - m_2)$ and compare it with the projection of $(m_1 + m_2)/2$: **linear discriminative**!

LDA: pros & cons

Pros:

- Effectively leverage label information to maximize class separability
- Effective for classification tasks

Cons:

- At most $C - 1$ feature projections
- Assume linear class boundaries
- Assume Gaussian-distributed data with equal covariance matrix (statistical perspective)
- Assume that different classes are identifiable through the mean

Quadratic Discriminant Analysis (QDA)

- **Why mention QDA here?**

  QDA is included only as a classification-side contrast to LDA
- **Key Difference from LDA:**

  QDA assumes each class has its own covariance matrix, which allows quadratic decision boundaries
- **Important distinction:**

  Unlike LDA, QDA is not a dimensionality-reduction method

  It models each class as a multivariate Gaussian with class-specific mean and covariance, and then predicts by maximum posterior probability

Stochastic Neighbor Embedding (SNE)

- **Goal:** Preserve the local neighborhood structure of high-dimensional data in a low-dimensional embedding
- **Idea:** Convert pairwise distances into probabilities in both spaces so nearby points receive most of the probability mass
- **Why probabilities?** Matching probabilities emphasizes neighborhood mistakes more than exact long-range distances

- **Key steps:**

1. Compute conditional probabilities $p_{j|i}$ in high-dimensional space
2. Define conditional probabilities $q_{j|i}$ in low-dimensional space
3. Minimize divergence between $p_{j|i}$ and $q_{j|i}$.

点i选点j做邻居的概率

SNE: model similarity with probability

- **High-dimensional** similarity (jump from j to i in the original space):

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2/2\sigma_i^2\right)}{\sum_{k\neq i}\exp\left(-\|x_i - x_k\|^2/2\sigma_i^2\right)} \text{ for } j \neq i, \quad p_{i|i} = 0$$

  - σ_i is a bandwidth parameter, representing the "size" of the neighborhood
  - Search σ_i such that $H(P_i) = -\sum_j p_{j|i}\log p_{j|i} = \log \text{Perplexity}$

- **Low-dimensional** similarity (jump from j to i in the embedded space):

$$q_{j|i} = \frac{\exp\left(-\|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2\right)}{\sum_{k\neq i}\exp\left(-\|\boldsymbol{y}_i - \boldsymbol{y}_k\|^2\right)} \text{ for } j \neq i, \quad q_{i|i} = 0$$

  - σ_i is set uniformly to $1/\sqrt{2}$ for all samples
  - Reduce the computational complexity
  - Avoid overfitting to the high-dimensional structure

SNE: measure distribution difference

- **Objective.** Minimize Kullback-Leibler (KL) divergence:

$$C = \sum_i \mathrm{KL}(P_i||Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

- Optimized via gradient descent:

$$\frac{\partial C}{\partial \boldsymbol{y}_i} = 2 \sum_j (\boldsymbol{y}_i - \boldsymbol{y}_j)(p_{i|j} - q_{i|j} + p_{j|i} - q_{j|i})$$

  - Each sample i is connected with other samples via springs
  - Force direction: $\boldsymbol{y}_i - \boldsymbol{y}_j$
  - Force magnitude: $p_{i|j} - q_{i|j} + p_{j|i} - q_{j|i}$, mismatch between high-dimensional similarity and low-dimensional similarity

Symmetric SNE (optional)

- In SNE, $p_{i|j}$ and $q_{i|j}$ are asymmetric

$$C = KL(P \mid\mid Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}, \quad q_{ij} = \frac{\exp(-\|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2)}{\sum_{k \neq l} \exp(-\|\boldsymbol{y}_k - \boldsymbol{y}_l\|^2)}$$

- For a symmetric one, $p_{ij} = p_{ji}, q_{ij} = q_{ji}$
- **Choice 1:** $p_{ij} = \dfrac{\exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2/2\sigma^2)}{\sum_{k \neq l} \exp(-\|\boldsymbol{x}_k - \boldsymbol{x}_l\|^2/2\sigma^2)}$
  - Con: for an outlier x_i, p_{ij} are extremely small for all j, so $\boldsymbol{y}_i$ has little effect on the cost function $\Rightarrow$ x_i may "not be properly projected"
- **Choice 2** (better?): $p_{ij} = (p_{i|j} + p_{j|i})/2n$, which ensures that $\sum_j p_{ij} > 1/2n$

$$\frac{\partial C}{\partial \boldsymbol{y}_i} = 4 \sum_j (p_{ij} - q_{ij})(\boldsymbol{y}_i - \boldsymbol{y}_j)$$

SNE: pros & cons

Pros:

- Preserves local structure effectively
- Flexible framework for extensions

Cons:

- **Crowding problem**: Points clump in low-dimensional space
- Computationally expensive for large datasets
- Sensitive to perplexity parameter

Crowding problem

- **Fact 1.**

  Assume we can have 11 samples that are mutually equidistant in a 10-dimensional space, there is no way to model this faithfully in a 2-dimensional map

- **Fact 2.**

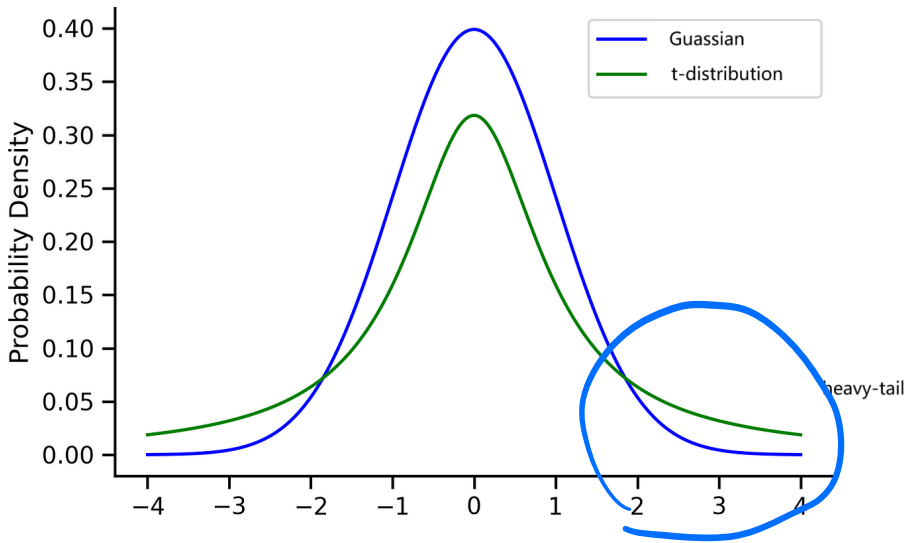
  The distribution of pairwise distances in high-D space and low-dimensional space is totally different: the volume of sphere $V(r) \propto r^D$, indicating that we don't have much room to place "neighbors" in low-D space

$\Rightarrow$ Samples with medium distance with sample i will be placed too far away (i.e., $p_{ij} > q_{ij}$), contributing to a large number of very small attractive forces. Thus, sample i will be placed at the center of the map

- **Solution.** Increase q_{ij} for distant points: we use t-distribution instead of the Gaussian distribution in the low-dimensional space

t-distribution

- ⋆ A **long-tail** distribution to convert distances into probabilities
- Allow a moderate distance in the high-dimensional space to be faithfully modeled by a much larger distance on the map
- Eliminate the unwanted attractive forces between map points that represent moderately dissimilar datapoints



t-Distributed SNE (t-SNE)

- Introduced by van der Maaten and Hinton (2008)
- An improvement over SNE that is mainly used for visualization of local structure

- **Key changes:**
  - Use a heavier-tailed Student's t-distribution in low-dimensional space
  - Symmetrize the objective function

- Widely used for visualizing high-dimensional data, but cluster sizes and distances between clusters should be interpreted cautiously

t-SNE: Mathematical Foundation

- High-dimensional similarity (symmetrized):

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

- Low-dimensional similarity (t-distribution, 1 degree of freedom):

$$q_{ij} = \frac{\left(1 + \|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

- **Objective.** Minimize KL divergence:

$$C = \mathrm{KL}(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- **optimization:** gradient descent with tricks (e.g., early exaggeration)

Early Exaggeration in t-SNE

- Technique to enhance cluster formation
- **Basic idea:** multiply high-dimensional similarities by a factor initially
- **Purpose:** help form tight, distinct clusters early during optimization

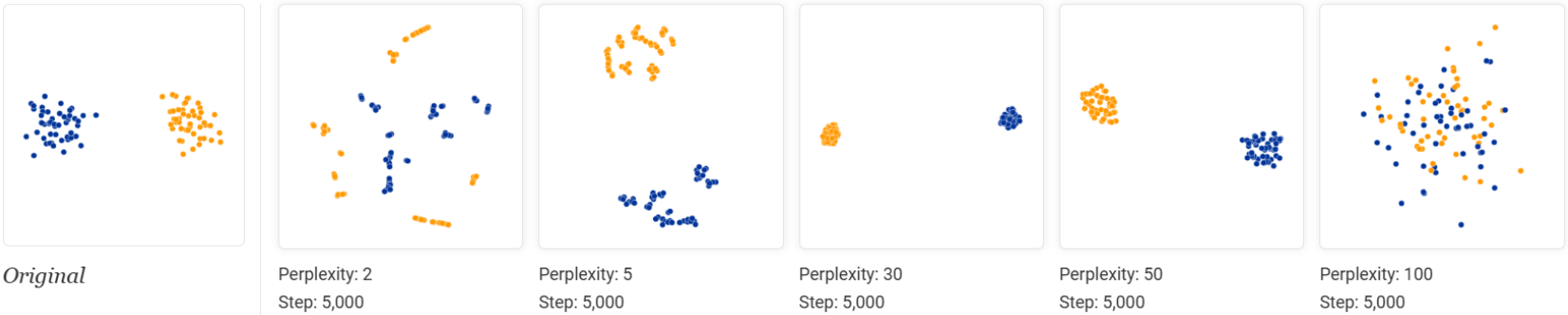
- **Process:**
  - Start with exaggerated similarities to pull similar points closer
  - After ~ 100 iterations, use true similarities to fine-tune positions

$\Rightarrow$ Results in clearer, more interpretable visualizations with well-defined clusters

- Reference:
  `https://blog.dailydoseofds.com/p/what-is-early-exaggeration-in-tsne`

How to use t-SNE Effectively

- Check this page: How to use t-SNE Effectively
- Those hyperparameters really matter



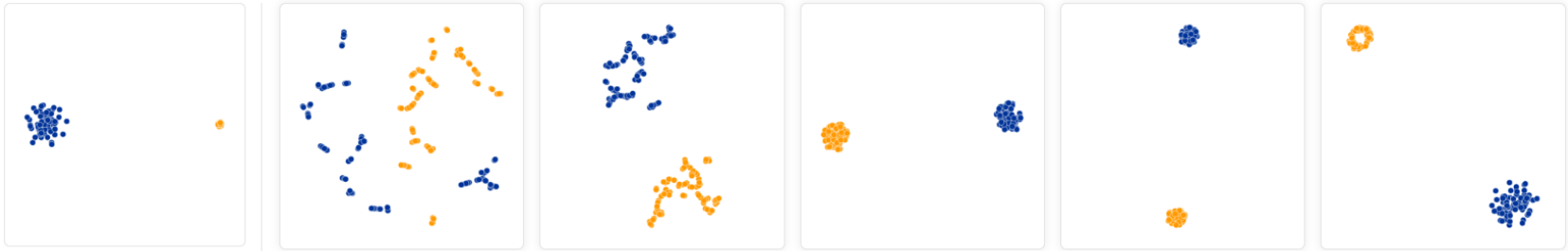
- **Perplexity:** controls the balance between local and global structure

1. Effective number of neighbors used to compute conditional probabilities
2. Low perplexity (e.g., 5–10): Emphasizes local structure, producing tight, fine-grained clusters but potentially missing global relationships
3. High perplexity (e.g., 30–50): Captures more global structure, spreading out points and potentially merging nearby clusters

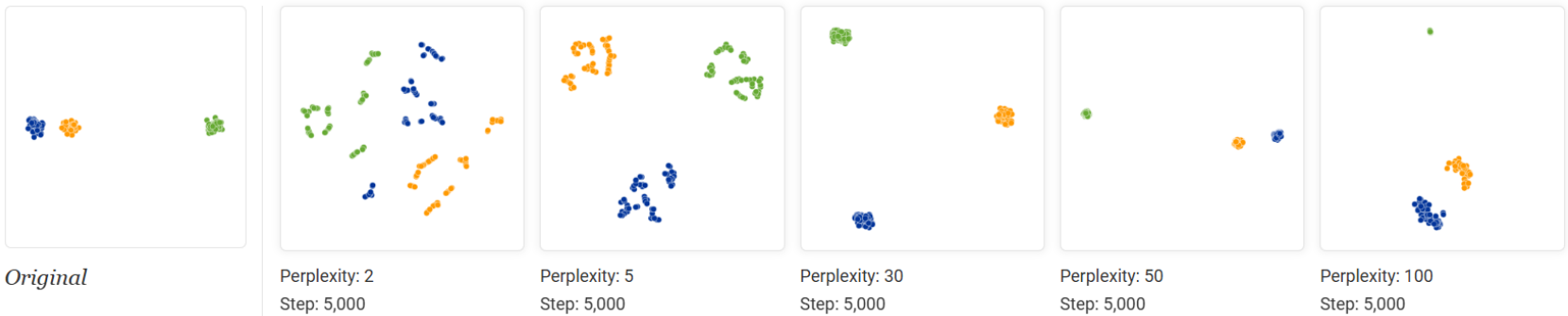
- **Learning rates:** step size during GD optimization of the KL divergence

How to use t-SNE Effectively

- Cluster sizes in a t-SNE plot mean nothing



- Distances between clusters might not mean anything



t-SNE: pros & cons

Pros:

- Excellent for visualizing clusters
- Mitigates crowding problem via t-distribution
- Robust to different datasets

Cons:

- Non-convex optimization (sensitive to initialization)
- Does not preserve global structure well
- Computationally intensive for large datasets (hours for 10k points)
- No easy way to embed new points

Uniform Manifold Approximation and Projection (UMAP)

- UMAP builds a weighted k-nearest-neighbor graph and learns a low-dimensional layout with similar connectivity
- Practical view:
  - `n_neighbors` controls the balance between local and global structure
  - `min_dist` controls how tightly points can pack in the embedding
  - The method is motivated by manifold and topological assumptions
- Key features:
  - Faster and more scalable than t-SNE
  - Usually preserves more global structure than t-SNE
  - Flexible for various distance metrics
  - Can embed new points

- `pip install umap-learn`

UMAP: learn the manifold structure

Construct a weighted k-neighbour graph. For each sample x_i

- We identify k nearest neighbors $N_k(x_i)$
- $\rho_i = \min\{d(x_i, x_j) \mid x_j \in N_k(x_i)\}$
- σ_i is set to make $\sum_{x_j \in N_k(x_i)} \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right) = \log_2 k$
- Asymmetric weighted k-neighbour graph $v_{j|i} = \exp\left(-\frac{d(x_i, x_j) - \rho_i}{\sigma_i}\right)$ for $x_j \in N_k(x_i)$. $v_{j|i} = 0$ otherwise
- Symmetric weighted k-neighbour graph: $B = A + A^\top - A \circ A^\top$, where A is the weighted adjacency matrix of the graph, i.e., $v_{ij} = v_{i|j} + v_{j|i} - v_{i|j} v_{j|i}$

UMAP: perform the projection

- **Minimize cross entropy**

$$C_{\text{UMAP}} = \sum_{i \neq j} v_{ij} \log \frac{v_{ij}}{w_{ij}} + (1 - v_{ij}) \log \frac{1 - v_{ij}}{1 - w_{ij}}$$

  - v_{ij} is the similarity in high-D space and was defined before
  - $w_{ij} = (1 + a\|y_i - y_j\|_2^{2b})^{-1}$ is the similarity in low-D space
- Can be solved using a force directed graph layout algorithm to project the graph
  - Iteratively applying attractive and repulsive forces at each edge or vertex
    - Attractive force: take derivative on $v_{ij} \log \frac{v_{ij}}{w_{ij}}$

$$-\frac{2ab\|\boldsymbol{y}_i - \boldsymbol{y}_j\|_2^{2b-2}}{1 + \|\boldsymbol{y}_i - \boldsymbol{y}_j\|_2^2} w(x_i, x_j)(\boldsymbol{y}_i - \boldsymbol{y}_j)$$

    - Repulsive force: take derivative on $(1 - v_{ij}) \log \frac{1-v_{ij}}{1-w_{ij}}$

$$\frac{2n}{(\varepsilon + \|\boldsymbol{y}_i - \boldsymbol{y}_j\|_2^2)(1 + \|\boldsymbol{y}_i - \boldsymbol{y}_j\|_2^2)} (1 - w(x_i, x_j))(\boldsymbol{y}_i - \boldsymbol{y}_j)$$

  - Optimized via stochastic gradient descent

UMAP: project unseen data

- UMAP allows the projection of unseen data
  - Query the k nearest neighbors for the input x
  - Calculate the weights
  - Use the weighted average on low-dimensional space $\sum w_i y_i / \sum w_i$ as the initial values
  - Optimize the position of new points while keeping other points fixed

- Why t-SNE does not support it?
  - The loss function $KL(P||Q)$ relies on all sample pairs
  - After adding new points, p_{ij} changes due to the changes in normalizing factors
  - If we fix the positions of existing samples, p_{ij} decreases while q_{ij} remains unchanged, making the optimization fail

UMAP: hyperparameters

- `n_neighbors`: number of nearest neighbors used to approximate the local manifold structure
  - It is analogous to perplexity in t-SNE
  - But directly specifies the neighbor count

- `min_dist`: minimum distance apart that points are allowed to be in the low-dimensional representation
  - Lower values allow points to cluster closely, producing compact, well-separated groups
  - Larger values will prevent UMAP from packing points together and will focus on the preservation of the broad topological structure instead

- Check this page umap-learn for more details and visualization examples

UMAP: pros & cons

Pros:

- Fast and scalable
- Preserves global and local structure

Cons:

- Sensitive to hyperparameters
- May overfit to noise in some cases

Comparison

Method	Linear/Non-Linear	Supervised	Key Use
PCA	Linear	No	Variance maximization
Classical MDS	Linear	No	Distance preservation
◦ Metric MDS	Non-Linear	No	Distance preservation
◦ ISOMAP	Non-Linear	No	Geodesic distances
LDA	Linear	Yes	Class separation
SNE	Non-Linear	No	Local structure
◦ t-SNE	Non-Linear	No	Visualization
◦ UMAP (optional)	Non-Linear	No	Visualization